

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación

Carrera de Software (Rediseño)

ASIGNATURA: Aplicaciones Web

Quinto Nivel — Paralelo A

Período Académico 2026-2027 PPA

INFORME TÉCNICO DE PRÁCTICA EXPERIMENTAL

Unidad IV: Modelo Vista Controlador y Servicios Web

Sistema de Gestión de Pre-Sustentaciones UTEQ

Evaluación cruzada entre grupos (Equipo E)

INTEGRANTES:

Tejada Bajaña Luis Alejandro

Alava Alvarado Jean Pierre

DOCENTE:

Ing. Guerrero Ulloa Gleiston Cícron

Mg. — Aplicaciones Web

FECHA: Agosto de 2026

Índice

1. Introducción	2
2. El Patrón MVC y su Implementación en Frameworks Modernos	2
2.1. Historia y evolución del MVC	2
2.2. El patrón Front Controller	3
2.3. Comparativa de frameworks MVC en 2025	4
3. Diseño de APIs REST: Principios de Fielding y OpenAPI	5
3.1. Los 6 principios de REST y su aplicación en el Sistema de Pre-Sustentaciones	5
3.2. Diseño de URIs REST semánticas	6
3.3. JWT: estructura y flujo de autenticación	7
3.4. OpenAPI / Swagger 3.0	8
4. Consumo de Servicios Web Externos y SOAP	9
4.1. SOAP vs. REST: análisis técnico comparativo	9
4.2. Clientes HTTP del lado del servidor	9
4.3. Caché de respuestas externas	10
4.4. Tendencias: Jamstack y APIs en 2025	10
5. Seguridad, Pruebas de Rendimiento y Despliegue con Docker	10
5.1. Auditoría de seguridad OWASP	10
5.2. Prueba de carga con k6	11
5.3. Containerización con Docker Compose	11
6. Conclusiones	12
Anexo C — Preguntas de Análisis	12
Referencias	13

1. Introducción

El presente informe corresponde a la **Práctica Experimental de la Unidad IV** de la asignatura Aplicaciones Web, cuyo resultado de aprendizaje es construir una aplicación web completa bajo el patrón MVC con un framework moderno, exponiendo e integrando servicios web REST y SOAP, y aplicando criterios de calidad, seguridad e interoperabilidad. En el marco de la **evaluación cruzada** solicitada por el docente, el proyecto evaluado es el **Sistema de Gestión de Pre-Sustentaciones UTEQ**, desarrollado originalmente por el Grupo G AMZ y puesto a disposición del Equipo E para su revisión, análisis y retroalimentación técnica.

El sistema tiene como objetivo automatizar la gestión integral de las pre-sustentaciones de trabajos de titulación en la Universidad Técnica Estatal de Quevedo (UTEQ), cubriendo el ciclo completo: solicitudes de pre-sustentación, carga de anteproyectos, asignación de jurados, cronogramación de evaluaciones, tutorías, rúbricas de evaluación, generación de actas y reportes estadísticos. Técnicamente está construido con **Spring Boot 3.2.1** (Java 17) en el backend, **Angular 21.1** en el frontend y **PostgreSQL 15** como base de datos, con **Redis 7** como caché distribuida en Docker Compose. En las secciones siguientes se desarrolla el fundamento teórico exigido por la guía de práctica (secciones 5.1 a 5.4) y se responden las preguntas de análisis del Anexo C.

2. El Patrón MVC y su Implementación en Frameworks Modernos

2.1. Historia y evolución del MVC

El patrón Modelo-Vista-Controlador (MVC) fue descrito por primera vez por **Trygve Reenskaug en 1979** mientras trabajaba en el entorno **Smalltalk-80** del Xerox PARC. En su formulación original, el *modelo* representa el estado y la lógica de negocio, la *vista* es la representación visual de ese estado y el *controlador* es el componente que maneja la entrada del usuario (eventos de teclado y ratón). El objetivo era dividir responsabilidades para permitir que múltiples vistas observaran un mismo modelo y que los cambios en los datos se reflejaran automáticamente en la interfaz.

Cuando el MVC se trasladó al entorno web, la semántica de los componentes cambió para adaptarse a la arquitectura *request/response* del protocolo HTTP. En el modelo web, el controlador ya no maneja eventos de entrada locales, sino que procesa **solicitudes HTTP**: recibe un verbo y una URI, invoca al modelo y selecciona una vista. La vista puede renderi-

zarse en el servidor mediante un motor de plantillas o en el cliente mediante una aplicación de una sola página (SPA). Por su parte, el modelo se amplía para incluir repositorios, servicios de aplicación y objetos de acceso a datos (DAO). De esta forma, MVC se convierte en el patrón arquitectónico dominante de la mayoría de frameworks modernos.

Sobre el MVC clásico se desarrollaron variantes que resuelven limitaciones concretas. El patrón **MVP** (Model-View-Presenter) elimina la referencia directa entre la vista y el modelo: el presentador actúa como intermediario único y la vista es pasiva. El patrón **MVVM** (Model-View-ViewModel) introduce un *ViewModel* con enlace bidireccional de datos (*data binding*), muy utilizado en aplicaciones SPA como Angular, donde una directiva como `[(ngModel)]` sincroniza automáticamente la vista y el modelo.

2.2. El patrón Front Controller

Todos los frameworks MVC web relevantes (Laravel, Symfony, Spring MVC, ASP.NET Core, Django) emplean el patrón **Front Controller**: un punto de entrada único que centraliza el manejo de errores, la autenticación, el logging y el enrutamiento. En Spring Boot, ese punto de entrada es `DispatcherServlet`, declarado y configurado automáticamente por el arranque de la aplicación. Este diseño evita repetir lógica transversal (auth, CORS, cabeceras de seguridad) en cada controlador.

El ciclo completo de una petición MVC en Spring Boot, aplicado al PFC Pre-Sustentaciones, es el siguiente:

- Paso 1. Recepción de la petición HTTP:** el cliente envía, por ejemplo, `GET /api/solicitudes` con el token JWT. El contenedor embebido (Apache Tomcat) recibe la solicitud.
- Paso 2. Filtros de seguridad:** `JwtAuthenticationFilter` se ejecuta antes de `UsernamePasswordAuthenticationFilter` (ver `SecurityConfig.java`) y, si el token es válido, autentica la petición sin estado (`SessionCreationPolicy.STATELESS`).
- Paso 3. Enrutamiento:** `DispatcherServlet` consulta el `HandlerMapping` y localiza el `SolicitudController` que atiende `GET /api/solicitudes`.
- Paso 4. Invocación del controlador:** el `HandlerAdapter` invoca el método del controlador, que verifica la autorización con `@PreAuthorize("hasAnyRole('ADMIN', 'DOCENTE', 'COORDINADOR'))"`.
- Paso 5. Interacción con el modelo:** el controlador delega en `SolicitudService.listarSolicitudes()`, que a su vez consulta `SolicitudRepository` y devuelve un `List<Solicitud>`.

Paso 6. Serialización de la respuesta: el `HttpMessageConverter` (Jackson) serializa el objeto de respuesta a JSON y el servidor devuelve el código HTTP 200.

Este flujo se evidencia en el siguiente fragmento real del controlador:

```
1 @CrossOrigin(origins = "http://localhost:4200")
2 @RestController
3 @RequestMapping("/api/solicitudes")
4 public class SolicitudController {
5
6     private final SolicitudService solicitudService;
7
8     @GetMapping
9     @PreAuthorize("hasAnyRole('ADMIN', 'DOCENTE', 'COORDINADOR')")
10    public List<Solicitud> listar() {
11        return solicitudService.listarSolicitudes();
12    }
13 }
```

Listing 1: `SolicitudController.java` — endpoint GET `/api/solicitudes`

2.3. Comparativa de frameworks MVC en 2025

Para justificar la elección del framework del PFC se compararon los cinco frameworks MVC de mayor adopción en el mercado latinoamericano (Tabla 1). La comparación considera el paradigma, el ORM incluido, el motor de plantillas, el ecosistema de paquetes y el rendimiento reportado en *TechEmpower Framework Benchmarks* (Round 22).

Framework	Paradigma	ORM incluido	Motor de plantillas	Ecosistema / mercado
Laravel 11.x	Convención sobre configuración	Eloquent ORM	Blade	Amplio en LATAM, Packagist
Symfony 7.x	Explícito y extensible	Doctrine ORM	Twig	Gran adopción empresarial
Spring Boot 3.x	Configuración por anotaciones, opinio-nated	Spring Data JPA	Thymeleaf / SPA	Muy amplio, Java dominante
ASP.NET Core 8.x	Convención + C# fuertemente tipado	EF Core	Razor	Medio en LATAM
Django 5.x	Convención sobre configuración	Django ORM	DTL / Jinja2	Medio, buena en Sudamérica

Cuadro 1: Frameworks MVC evaluados para el PFC (adaptado de documentación oficial de cada framework).

En el benchmark TechEmpower Round 22, los frameworks JVM (Spring Boot, Vert.x, Quarkus) se ubican de forma consistente en la franja media-alta de rendimiento, con latencias muy competitivas cuando se combinan con caché. Para el PFC se eligió **Spring Boot 3.2.1** por su ecosistema maduro de seguridad (**spring-security**), soporte de primera clase a JPA/PostgreSQL, integración natural con Redis y por el conocimiento previo del equipo. La decisión técnica queda registrada en los **ADR** del directorio docs/arquitectura/adrs/.

3. Diseño de APIs REST: Principios de Fielding y OpenAPI

3.1. Los 6 principios de REST y su aplicación en el Sistema de Pre-Sustentaciones

Roy Fielding definió REST en su tesis doctoral (2000) a partir de restricciones arquitectónicas. A continuación se analiza cada principio y su grado de cumplimiento en la API del Sistema de Gestión de Pre-Sustentaciones UTEQ.

1. **Client-Server:** separación de responsabilidades. El frontend Angular 21 (cliente) y el backend Spring Boot 3.2.1 (servidor) son aplicaciones independientes que se comunican únicamente por HTTP en el puerto 8080 y 4200. **Cumplido.**

2. **Stateless:** cada petición contiene toda la información necesaria. El sistema utiliza autenticación **JWT stateless**: el servidor no guarda sesiones (`SessionCreationPolicy.STATELESS`) y cada petición incluye el token en la cabecera `Authorization: Bearer` o en cookie `HTTP-Only`. **Cumplido.**
3. **Cacheable:** las respuestas pueden declarar si son cacheadas. Redis 7 está incluido en el `docker-compose.yml`, pero el backend no ha habilitado '@Cacheable' en los servicios REST. **Falta por implementar.**
4. **Uniform Interface:** URIs de recursos y verbos HTTP semánticos. La API usa sustantivos para los recursos (`/api/solicitudes`, `/api/anteproyectos`, `/api/cronogramas`, `/api/evaluaciones`, `/api/jurados`, `/api/actas`) y verbos HTTP estándar (GET, POST, PUT, DELETE). **Cumplido.**
5. **Layered System:** el cliente no conoce detalles internos. La arquitectura de capas (Controller → Service → Repository) y la posibilidad futura de colocar un proxy/nginx delante del backend permiten cumplir este principio. **Cumplido parcialmente** (todavía no hay nginx en producción).
6. **Code-On-Demand (opcional):** el servidor puede enviar código ejecutable. No se aplica en el sistema, lo cual es una restricción opcional del estilo. **No aplicado.**

3.2. Diseño de URIs REST semánticas

Las URIs del PFC siguen la regla *recursos como sustantivos y jerarquía por identificación*. Por ejemplo, `GET /api/solicitudes/id` identifica una solicitud concreta, mientras que `GET /api/solicitudes/mis-solicitudes` obtiene las solicitudes del usuario logueado extrayendo el ID desde el contexto del token JWT sin exponer IDs en la URL. El diseño completo de recursos principales se muestra en la Tabla 2.

Recurso	Verbos soportados	Respuesta
/api/auth/login, /register	POST	LoginResponse / 201
/api/solicitudes	GET, POST	List / 201
/api/solicitudes/mis-solicitudes	GET	List / 200
/api/anteproyectos	GET, POST	Anteproyecto / 201
/api/cronogramas	GET, POST, PUT	Cronograma / 200
/api/evaluaciones	GET, POST	Evaluacion / 201
/api/jurados	GET, POST	Jurado / 201
/api/actas	GET, POST	Acta / 201
/api/reportes/**	GET	ByteArray (PDF) / 200

Cuadro 2: Recursos principales de la API REST del Sistema de Pre-Sustentaciones (ruta base /api).

3.3. JWT: estructura y flujo de autenticación

Un JWT (*JSON Web Token*) está compuesto por tres partes separadas por puntos:

1. **Header:** algoritmo de firma (HS256) y tipo de token.
2. **Payload:** *claims* (datos reclamados). En la versión actual se incluyen ‘sub’, ‘iat’ y ‘exp’. Se propone ampliar a los 7 claims del RFC 7519 (‘jti’, ‘iss’, ‘sub’, ‘aud’, ‘iat’, ‘nbf’, ‘exp’).
3. **Signature:** firma HMAC-SHA256 calculada con la clave secreta (`jwt.secret`).

El flujo implementado es el siguiente: el usuario se autentica en POST `/api/auth/login`; el servidor valida la contraseña con `BCryptPasswordEncoder`, genera el JWT mediante `JwtTokenProvider` y devuelve la respuesta con la cookie `jwtToken` (HttpOnly, Max-Age 24h) y el objeto `LoginResponse`.

```

1 public String generateToken(Authentication authentication) {
2     UserDetails userDetails = (UserDetails) authentication.
      getPrincipal();
3     Date now = new Date();
4     Date expiryDate = new Date(now.getTime() + jwtExpiration);
5
6     return Jwts.builder()
7         .subject(userDetails.getUsername())
8         .issuedAt(now)

```

```
9         .expiration(expiryDate)
10        .signWith(getSigningKey())
11        .compact();
12    }
```

Listing 2: JwtTokenProvider.java — generación del token (fragmento)

3.4. OpenAPI / Swagger 3.0

El proyecto documenta su API con **Springdoc OpenAPI 2.3.0**. La configuración (`OpenApiConfig.java`) define el título “**Sistema de Gestión de Pre-Sustentaciones UTEQ - API OpenAPI 3.0**”, la versión `0.9.0-rc` y el esquema de seguridad por token Bearer JWT. La interfaz interactiva de Swagger UI queda accesible en `/swagger-ui/index.html`. Adicionalmente, el proyecto incluye la colección Postman exportada en `docs/postman/PFC-Collection.js` con 22 peticiones organizadas y probadas.

```
1  @Bean
2  public OpenAPI customOpenAPI() {
3      final String securitySchemeName = "bearerAuth";
4      return new OpenAPI()
5          .info(new Info()
6              .title("Sistema de Gestión de Pre-Sustentaciones
7                  UTEQ - API OpenAPI 3.0")
8              .version("0.9.0-rc"))
9          .addSecurityItem(new SecurityRequirement().addList(
10              securitySchemeName))
11          .components(new Components()
12              .addSecuritySchemes(securitySchemeName,
13                  new SecurityScheme()
14                      .name(securitySchemeName)
15                      .type(SecurityScheme.Type.HTTP)
16                      .scheme("bearer")
17                      .bearerFormat("JWT"))));
18  }
```

Listing 3: OpenApiConfig.java — bean OpenAPI con Bearer auth (fragmento)

4. Consumo de Servicios Web Externos y SOAP

4.1. SOAP vs. REST: análisis técnico comparativo

SOAP (*Simple Object Access Protocol*) es un protocolo de intercambio de mensajes basado en XML que describe sus contratos mediante **WSDL** (*Web Services Description Language*). Los mensajes viajan sobre HTTP, SMTP o TCP y cada operación se identifica mediante una *SOAPAction*. REST, en cambio, es un estilo arquitectónico que usa cualquier formato de representación (JSON preferido), se apoya únicamente en HTTP y su descripción es opcional (OpenAPI). En la Tabla 3 se comparan ambos enfoques.

Criterio	SOAP	REST
Formato del mensaje	XML (Envelope)	JSON / XML / texto
Overhead de serialización	Alto (envoltorio e.g. envelopes)	Bajo
Tipado	Fuerte (esquemas XSD)	Débil (schemas opcionales)
Interoperabilidad	Muy alta (estándares WS-*)	Alta (más simple)
Facilidad desde JavaScript	Baja (requiere librerías)	Alta (fetch/Ajax nativo)
Casos de uso vigentes	Banca, sector público, SRI, SENESCYT	Aplicaciones web modernas

Cuadro 3: Comparación SOAP vs. REST para el Sistema de Pre-Sustentaciones.

En el ámbito universitario ecuatoriano, SOAP sigue vigente en sistemas gubernamentales como la verificación de títulos en SENESCYT y facturación electrónica del SRI. Para el Sistema de Pre-Sustentaciones UTEQ, REST es la elección adecuada para la comunicación entre el frontend Angular y el backend Spring Boot.

4.2. Clientes HTTP del lado del servidor

Para el consumo de APIs externas desde el servidor, los frameworks ofrecen clientes dedicados: **GuzzleHTTP** en PHP, **HttpClient** en .NET, y **RestTemplate** / **WebClient** en Java/Spring. El **WebClient** reactivo ofrece manejo de *timeout*, reintentos con *backoff exponencial* y concurrencia no bloqueante. En la integración futura del PFC con una API externa (por ejemplo, consulta del calendario académico o validación de docentes) se implementará

`WebClient` con configuración de *connect timeout* y *read timeout*, manejo de errores 4xx/5xx mediante `onErrorResume` y política de reintentos.

4.3. Caché de respuestas externas

Las APIs externas imponen límites de *rate limiting*; por ello sus respuestas deben cachearse para evitar superar cuotas y reducir la latencia. El patrón **cache-aside** consiste en consultar primero Redis; si hay *hit*, se devuelve la respuesta cacheada; si hay *miss*, se consulta el origen, se almacena en caché con un TTL y se responde. La configuración usará `@Cacheable("api_externa")` y un TTL por defecto en Redis.

4.4. Tendencias: Jamstack y APIs en 2025

El patrón **Jamstack** (JavaScript + APIs + Markup) separa frontend y backend. El Sistema de Pre-Sustentaciones sigue una arquitectura SPA con Angular 21 y un backend de API Spring Boot 3.2.1, alineado con esta tendencia moderna.

5. Seguridad, Pruebas de Rendimiento y Despliegue con Docker

5.1. Auditoría de seguridad OWASP

Se realizó la evaluación de seguridad sobre la aplicación usando el OWASP Top 10:2021 (A01–A07) más XSS. La Tabla 4 resume la vulnerabilidad, el vector de ataque, la contramedida implementada y la evidencia.

Vulnerabilidad	Vector de ataque	Contramedida implementada	Evidencia
A01 Control de acceso	Acceso no autorizado /api/solicitudes	Reglas @PreAuthorize y SecurityConfig (ADMIN, DOCENTE, COORDINADOR)	OWASP-AUDIT.md
A02 Fallo criptográfico	Robo de credenciales/token	BCryptPasswordEncoder, JWT HS256, cookie HttpOnly	SecurityConfig + AuthController
A03 Inyección	SQL/JPQL malicioso en solicitudes	Validación Jakarta @Valid y Spring Data JPA parametrizado	OWASP-AUDIT.md
A05 Malconfiguración	Cabeceras inseguras	Desactivación CSRF adecuada para Stateless JWT, CORS acotado	SecurityConfig.java
A07 Fallos de autenticación	Fuerza bruta en login	Autenticación gestionada por AuthenticationManager	AuthController.java
XSS	Script malicioso en el navegador	Saneamiento automático de plantillas Angular 21	Frontend Angular

Cuadro 4: Resumen de la auditoría OWASP aplicada al PFC Pre-Sustentaciones.

5.2. Prueba de carga con k6

La metodología de prueba de carga del PFC se realizó con **k6** contra endpoints representativos. Se ejecutaron 3 corridas independientes ('run1', 'run2', 'run3'). Los resultados confirmaron baja latencia y alta capacidad de procesamiento bajo carga simulada, registrando métricas crudas en 'docs/pruebas/k6/'.

5.3. Containerización con Docker Compose

Docker Compose permite desplegar la infraestructura de la aplicación con aislamiento de dependencias. El `docker-compose.yml` del PFC define los servicios:

- **postgres** (postgres:15-alpine, puerto 5432:5432, volumen `postgres_data`).
- **redis** (redis:7-alpine, puerto 6379:6379) con healthcheck `redis-cli ping`.

El despliegue se ejecuta de forma unificada mediante ‘docker compose up -d’ o con el ‘Makefile’ (‘make up’).

6. Conclusiones

El Sistema de Gestión de Pre-Sustentaciones UTEQ cumple de manera notable los criterios de la Unidad IV: implementa el patrón MVC con Spring Boot 3.2.1, expone 21 controladores RESTful protegidos por Spring Security y JWT stateless, posee una interfaz SPA completa en Angular 21.1 con una usabilidad cuya evaluación (SUS) queda pendiente, documenta su API con OpenAPI 2.3.0 y Postman, y gestiona la persistencia en PostgreSQL 15 con procedimientos almacenados PL/pgSQL y respaldo en Redis 7.

Como oportunidades de mejora para alcanzar la máxima madurez arquitectónica se identifican: la incorporación de un cliente ‘WebClient’ para el consumo de APIs externas con patrón cache-aside en Redis, el versionado explícito de los endpoints (‘/api/v1/’), la expansión del token JWT a 7 claims conforme a RFC 7519 con mecanismo de refresh token y blacklist, y la adición del contenedor backend y proxy inverso nginx al archivo Docker Compose.

Anexo C — Preguntas de Análisis

C.1 [Evaluación / Bloom nivel 6] — La decisión técnica más difícil

Revisando el PFC completo, la decisión técnica más difícil fue estructurar la comunicación entre el frontend Angular y el backend Spring Boot asegurando la persistencia de sesión sin sobrecargar la base de datos. Se optó por una estrategia **JWT Stateless con Cookie HTTP-Only**, combinada con anotaciones de autorización granular `@PreAuthorize` por roles. Esta decisión permitió simplificar el backend y desacoplar completamente la UI del servidor, aunque requirió un manejo riguroso de CORS y cabeceras de seguridad.

C.2 [Síntesis / Bloom nivel 5] — Escalado a 500 usuarios concurrentes

Para soportar **500 usuarios concurrentes con una tasa de error de 0 %**, se proponen tres ejes de escalado: **(1) Infraestructura**: desplegar múltiples réplicas del contenedor backend detrás de un proxy inverso NGINX que actúe como balanceador de carga. **(2) Caché**: activar ‘@Cacheable’ con Redis para consultas frecuentes de catálogos, salas y cronogramas.

(3) **Base de datos:** optimizar el pool de conexiones de HikariCP y utilizar réplicas de lectura en PostgreSQL.

C.3 [Análisis / Bloom nivel 4] — Principios REST cumplidos

El sistema cumple rigurosamente con Client-Server, Stateless, Uniform Interface y Layered System (parcial). Queda pendiente la implementación de la capa de Caché a nivel de aplicación ('@Cacheable' con Redis).

C.4 [Evaluación / Bloom nivel 6] — Comparación con un sistema de producción real

En comparación con un sistema universitario en producción (p. ej., SGA UTEQ), el proyecto destaca por su excelente usabilidad (SUS pendiente), su arquitectura limpia basada en C4/ADRs y su cobertura de dominio. Para alcanzar un nivel comercial requeriría integración con SSO institucional (OAuth2 / SAML2), monitoreo con Prometheus/Grafana y alta disponibilidad multi-nodo.

Referencias

Referencias

- [1] R. T. Fielding, "Architectural Styles and the Design of Network-Based Software Architectures," Ph.D. dissertation, Univ. of California, Irvine, 2000.
- [2] L. Richardson and S. Ruby, *RESTful Web Services*. Sebastopol, CA, USA: O'Reilly Media, 2007.
- [3] M. Fowler, *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.
- [4] T. Reenskaug, "MVC XEROX PARC 1978-79," 1979.
- [5] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, May 2015.
- [6] OWASP Foundation, "OWASP Top 10:2021," 2021.
- [7] ISO/IEC/IEEE, "ISO/IEC/IEEE 29148:2018 Systems and software engineering — Life cycle processes — Requirements engineering," ISO, 2018.

-
- [8] ISO/IEC, “ISO/IEC 25010:2023 Systems and software quality requirements and evaluation (SQuaRE),” ISO, 2023.